

Announcements

- PA2 out, team contract due this Friday 2/7
 - Actual PA due 3/7
- HW2 due Weds 2/5
- In-class exam 1 next Weds, Feb 12 - 9:30-10:45a
 - Practice exams are on canvas - format may change, but content will be the same
 - One letter-sized (8.5 x 11") crib sheet (front and back) allowed
 - We will collect them after, so if you want to remember what was on it, please take a photo on your own beforehand

Exam 1 topics

- Efficiency / speedup / scalability analysis
- Analysis of multithreaded algorithms (work / span)
- Cache complexity analysis / cache-oblivious analysis
- Prefix sums

CSE 6220/CX 4220
Introduction to HPC

**Lecture 8: Applications of
Prefix Sums**

Helen Xu
hxu615@gatech.edu



Georgia Tech College of Computing
School of Computational
Science and Engineering

Recall: Prefix Sum (Scan) Definition

Input: an array $x = [x_0, x_1, \dots, x_{n-1}]$ of n elements

Output: an array $y = [y_0, y_1, \dots, y_{n-1}]$ of running sums, where

$$y_k = \begin{cases} x_0 & \text{if } k = 0 \\ x_k \oplus y_{k-1} & \text{if } k \geq 1. \end{cases}$$

Any binary associative operator (e.g., addition, multiplication, etc.)

A Partial List of Applications for Parallel Prefix

- Adding two n -bit integers in $O(\log n)$ time
- Evaluating polynomials
- Solving recurrences
- Radix sort
- “2D parallel prefix” for image segmentation
- Traversing linked lists
- and many others!

Application: Fibonacci via Matrix Multiply Prefix

$$F_{n+1} = F_n + F_{n-1}$$

$$\begin{pmatrix} F_{n+1} \\ F_n \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} F_n \\ F_{n-1} \end{pmatrix}$$

Can compute all F_n by `matmul_prefix` on

Select the upper
left entry

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}, \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}, \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}, \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}, \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \dots$$

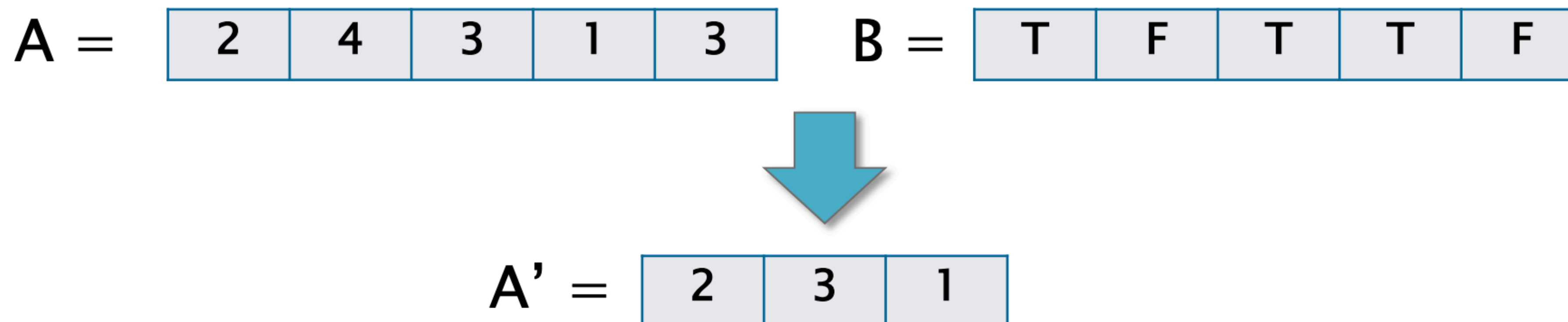
↓

$$\begin{pmatrix} 2 & 1 \\ 1 & 1 \end{pmatrix}, \begin{pmatrix} 3 & 2 \\ 2 & 1 \end{pmatrix}, \begin{pmatrix} 5 & 3 \\ 3 & 2 \end{pmatrix}, \begin{pmatrix} 8 & 5 \\ 5 & 3 \end{pmatrix}, \begin{pmatrix} 13 & 8 \\ 8 & 5 \end{pmatrix} \dots$$

The same idea works for any linear recurrence (e.g., random number generation).

Application: Stream Compression (aka Filter)

- Definition: Given a sequence $A=[x_0, x_1, \dots, x_{n-1}]$ and a Boolean array of flags $B[b_0, b_1, \dots, b_{n-1}]$, output an array A' containing just the elements $A[i]$ where $B[i] = \text{true}$ (maintaining relative order)
- Example:



- Can you implement filter using prefix sum?

Filter Implementation

Can use to filter on some condition

A =

2	4	3	1	3
---	---	---	---	---

B =

T	F	T	T	F
---	---	---	---	---

1	0	1	1	0
---	---	---	---	---

```
// Assume B'[n] = total sum  
parallel-for i=0 to n-1:  
    if(B'[i] != B'[i+1]):  
        A'[B'[i]] = A[i];
```



Prefix sum

B' =

0	1	1	2	3
---	---	---	---	---

Total sum = 3

Allocate array of size 3

--	--	--



A' =

2	3	1
---	---	---

Application: Radix Sort (Serial)

4	7	2	6	3	5	1	0
---	---	---	---	---	---	---	---

4	2	6	0	7	3	5	1
---	---	---	---	---	---	---	---

$b_0 = 0$

$b_0 = 1$

Sort on least significant bit (b_0 in $b_2b_1b_0$)
 $XX0 < XX1$ (evens before odds)

4	2	6	0	7	3	5	1
---	---	---	---	---	---	---	---

4	0	5	1	2	6	7	3
---	---	---	---	---	---	---	---

$b_1 = 0$

$b_1 = 1$

Stably sort entire array on next bit
 $X0X < X1X$

4	0	5	1	2	6	7	3
---	---	---	---	---	---	---	---

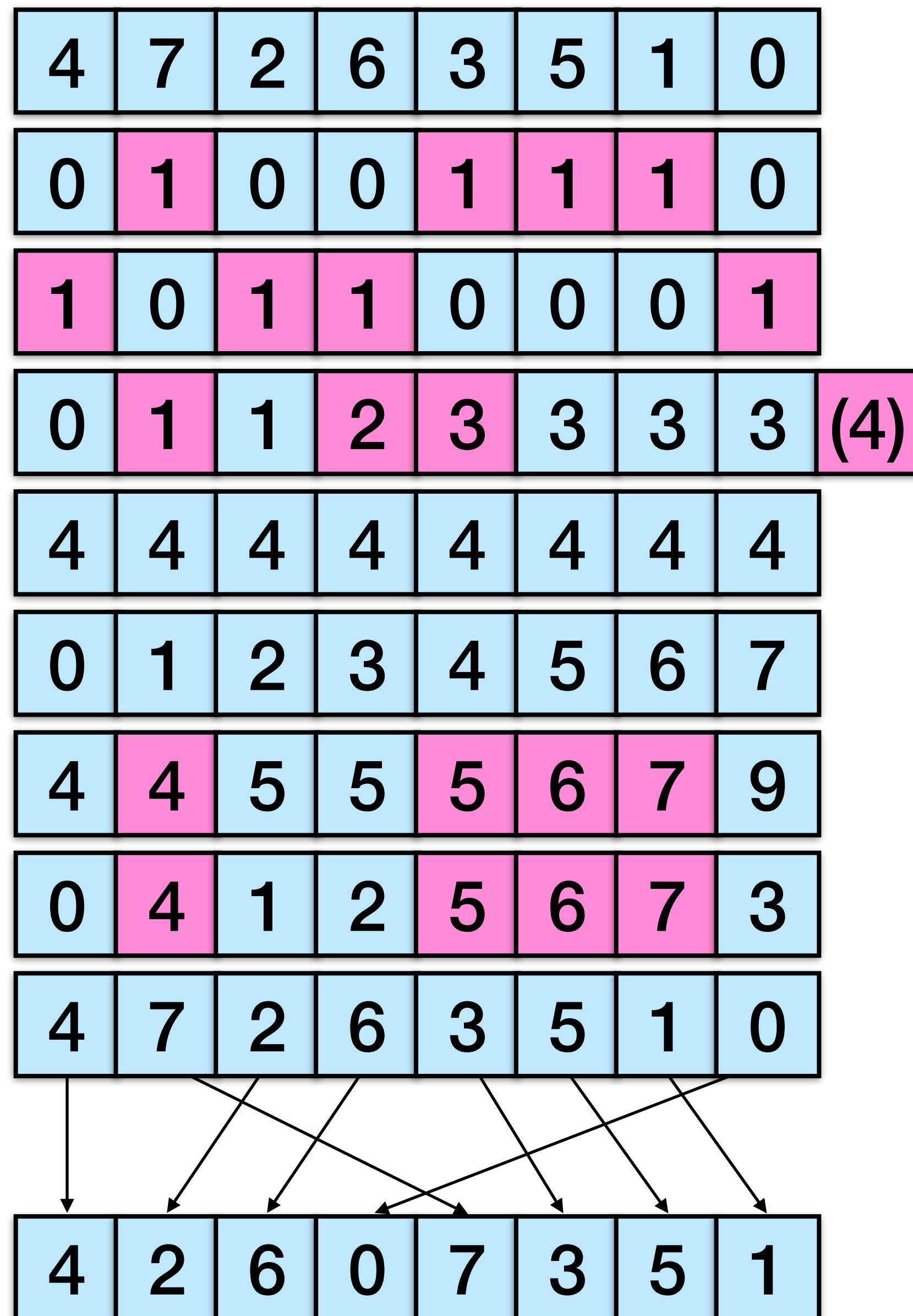
0	1	2	3	4	5	6	7
---	---	---	---	---	---	---	---

$b_2 = 0$

$b_2 = 1$

Stably sort entire array on next bit
 $0XX < 1XX$

Application: Data-Parallel Radix Sort



Input

Odds = last bit of each element

Evens = complement of odds

Even_positions = **exclusive scan** of evens

totalEvens = broadcast last element

index = constant array of 0 .. n

odd_positions = #evens + indx - even_pos

pos = get positions using masked assignment

Scatter input according to pos

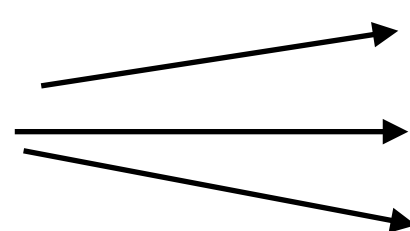
(repeat with next bit until you are out of bits)

Analyzing Data-Parallel Radix Sort

ALGORITHM: RADIX_SORT(A, b)

```
1  for  $i$  from 0 to  $b - 1$ 
2    FLAGS :=  $\{(a \gg i) \bmod 2 : a \in A\}$ 
3    NOTFLAGS :=  $\{1 - b : b \in \text{FLAGS}\}$ 
4     $R_0$  := SCAN(NOTFLAGS)
5     $s_0$  := SUM(NOTFLAGS)
6     $R_1$  := SCAN(FLAGS)
7     $R$  :=  $\{\text{if } \text{FLAGS}[j] = 0 \text{ then } R_0[j] \text{ else } R_1[j] + s_0 : j \in [0..|A|)\}$ 
8     $A := A \leftarrow \{(R[j], A[j]) : j \in [0..|A|)\}$ 
9  return  $A$ 
```

$O(\log n)$ span



What is the work and span?

Work: There are b iterations of the for loop, and iteration takes $O(n)$ work, so the total work is $O(bn)$.

Span: There are b iterations of the for loop, and iteration has $O(\log n)$ span, so the total span is $O(b \log n)$.

Application: Adding n-bit Integers

Problem: Computing sum of two n-bit binary numbers, a and b.

$$a = a_{n-1}a_{n-2}\dots a_0 \text{ and } b = b_{n-1}b_{n-2}\dots b_0$$

$$s = a + b = s_n s_{n-1} \dots s_0 \text{ (using carry bit array } c = c_{n-1}, \dots, c_0, c_{-1})$$

```
c[-1] = 0 // rightmost carry bit
for i = 0 to n - 1: //compute right to left
    s[i] = (a[i] xor b[i]) xor c[i-1] // one or three 1s
    c[i] = ((a[i] or b[i]) and c[i-1]) or (a[i] and b[i]) // next carry bit
```

Example:

a = 22

b = 29

a	1 0 1 1 0	←	s[0] depends on these
b	1 1 1 0 1	←	
c	1 1 1 0 0 0 0	←	
s	1 1 0 0 1 1		

Goal: Compute all c_i in $O(\log n)$ span via parallel prefix

Application: Adding n-bit Integers

```
c[-1] = 0 // rightmost carry bit
for i = 0 to n - 1: //compute right to left
    c[i] = ((a[i] or b[i]) and c[i-1]) or (a[i] and b[i]) // next carry bit
```

Idea: Split carry bit into two cases that indicate information about the carry-out (c_i) regardless of carry-in (c_{i-1}):

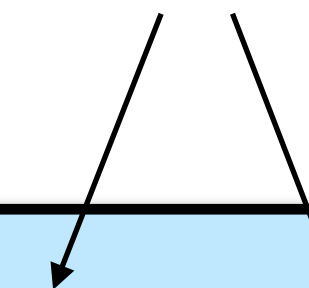
- Generate (g_i): This column will generate a carry-out **whether or not** the carry-in is 1.

$$g_i = a_i \& \& b_i$$

- Propagate (p_i): This column will propagate a carry-in **if there is one** to the carry-out.

$$p_i = a_i || b_i$$

can be computed in parallel


$$c_i = g_i || (p_i \& \& c_{i-1})$$

Carry Lookahead Logic

Idea: Define each carry-in in terms of p_i , g_i and the initial carry in c_{i-1} and not in terms of carry chain (i.e., unwind the recursion):

- $c_0 = g_0 + p_0 c_{-1}$
- $c_1 = g_1 + p_1 c_0 = g_1 + p_1 g_0 + p_1 p_0 c_{-1}$
- ...

Can be expressed with 2-by-2 boolean matrix multiplication:

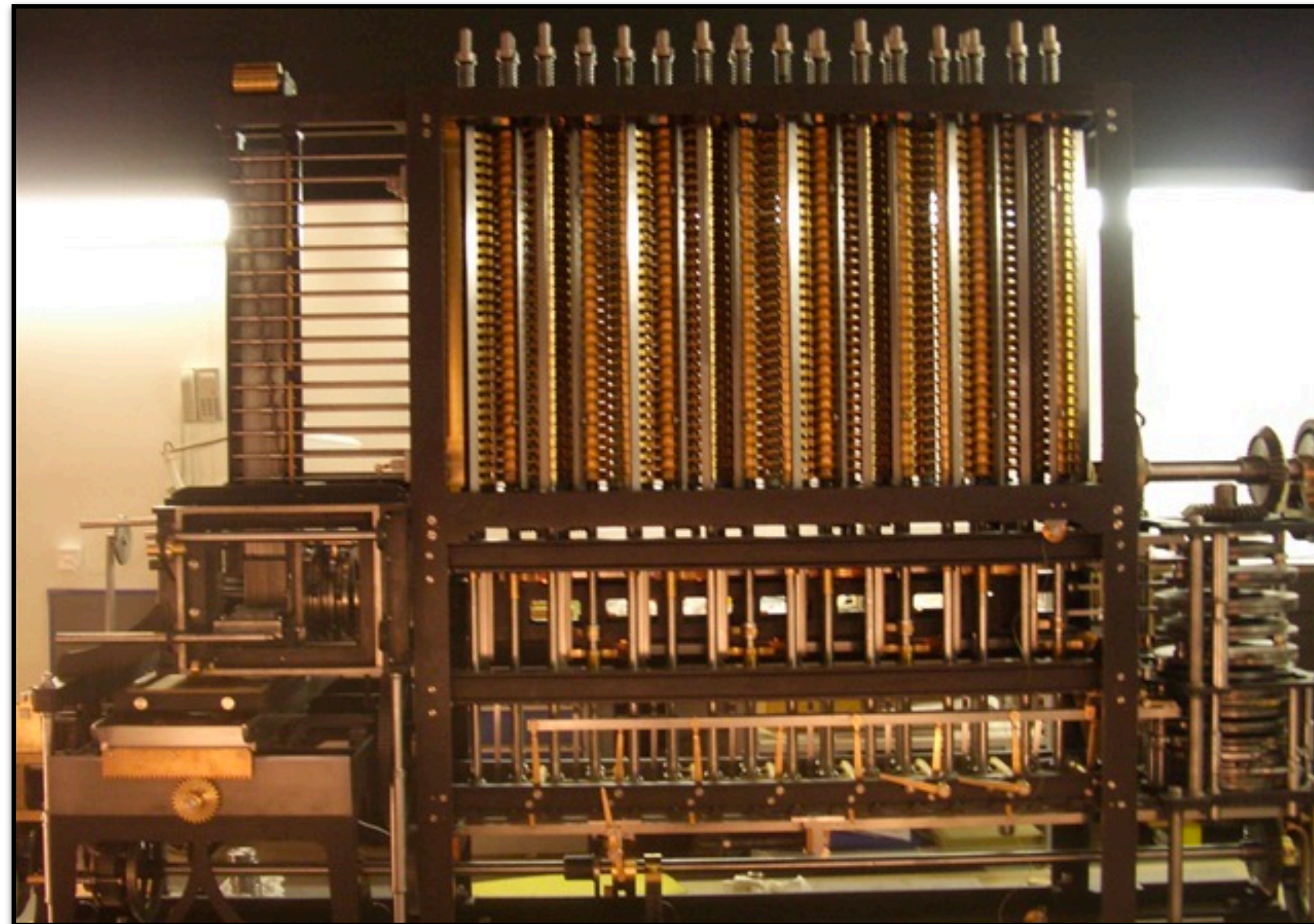
$$M[i] = \begin{pmatrix} p[i] & g[i] \\ 0 & 1 \end{pmatrix}$$

$$\begin{pmatrix} c[i] \\ 1 \end{pmatrix} = M[i] \times M[i-1] \times \dots \times M[0] \times \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

Carry-lookahead addition is used in all computers

This idea is used in all hardware

The design goes back to Babbage in the 1800s:



Application: Lexical Analysis

Lexical analysis **divides a long string of characters into tokens** - often the first thing a compiler does when processing a program.

Suppose we have a regular language - we can represent it with a **finite-state automaton** that begins in a certain state and makes transitions between states based on the characters read.

```
if x <= n then print ( "x = " , x ) ;
```

TABLE I. A Finite-State Automaton for Recognizing Tokens

Old State	Character Read													New line
	•	A	B	...	Y	Z	+	-	*	<	>	=	"	Space
N		A	A	...	A	A	*	*	*	<	<	*	Q	N
A		Z	Z	...	Z	Z	*	*	*	<	<	*	Q	N
Z		Z	Z	...	Z	Z	*	*	*	<	<	*	Q	N
*		A	A	...	A	A	*	*	*	<	<	*	Q	N
<		A	A	...	A	A	*	*	*	<	<	=	Q	N
=		A	A	...	A	A	*	*	*	<	<	*	Q	N
Q		S	S	...	S	S	S	S	S	S	S	S	E	S
S		S	S	...	S	S	S	S	S	S	S	S	E	S
E		E	E	...	E	E	*	*	*	<	<	*	S	N

Seems to depend on the previous state, which depends on characters read up to some point

Goal: Perform lexical analysis in parallel

Application: Lexical Analysis

Idea: replace every character in the string with the array representation of its **state-to-state function** (column).

Then perform a parallel-prefix operation with $\oplus$ as the **array composition**. Each character becomes an array representing the **state-to-state function** for that prefix.

Use the **initial state** to index into the arrays.

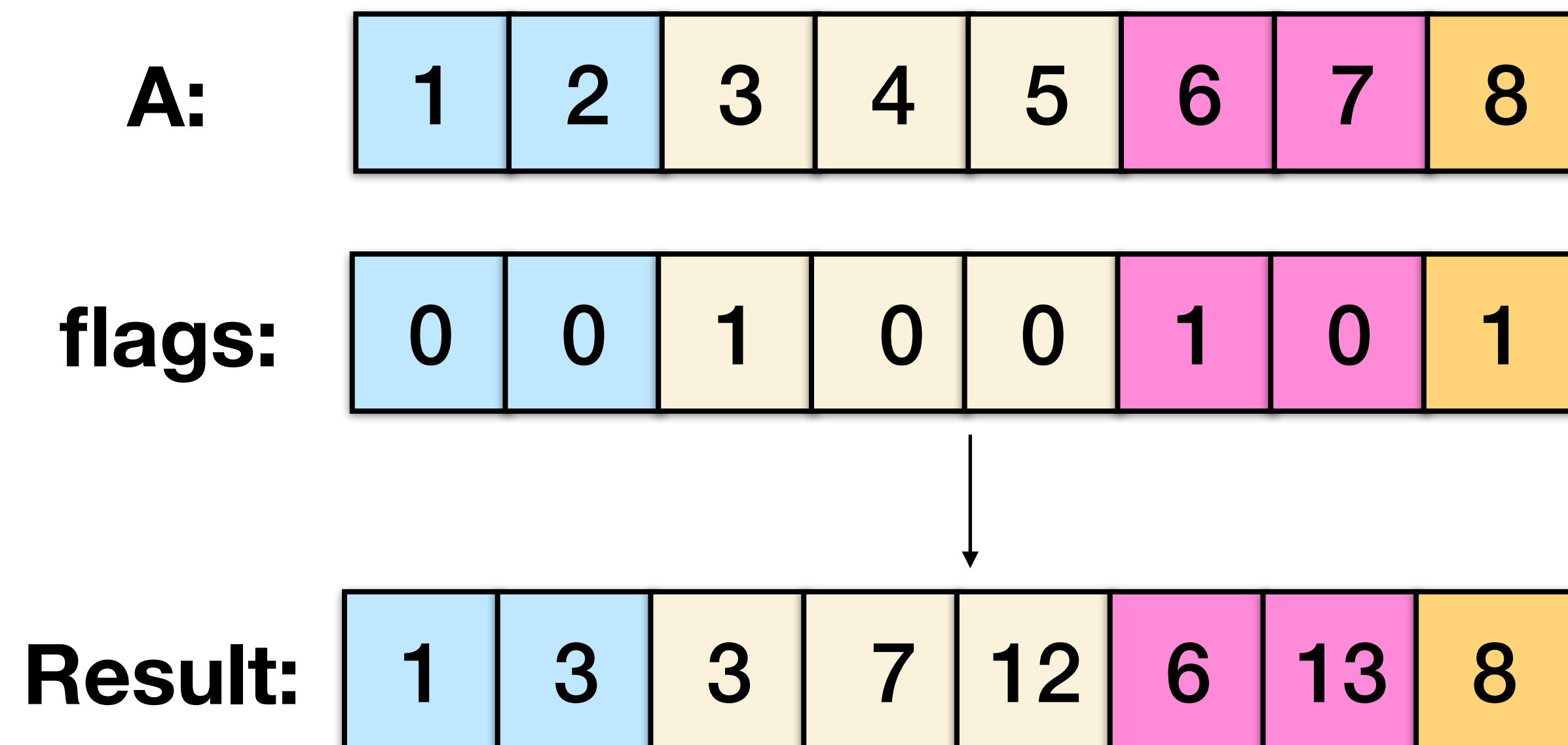
Example of state-to-state function for the space character

TABLE I. A Finite-State Automaton for Recognizing Tokens

Old State	Character Read													New line
	•	A	B	...	Y	Z	+	-	*	<	>	=	"	
N	•	A	A	...	A	A	*	*	*	<	<	*	Q	N
A		Z	Z	...	Z	Z	*	*	*	<	<	*	Q	N
Z		Z	Z	...	Z	Z	*	*	*	<	<	*	Q	N
*		A	A	...	A	A	*	*	*	<	<	*	Q	N
<		A	A	...	A	A	*	*	*	<	<	=	Q	N
=		A	A	...	A	A	*	*	*	<	<	*	Q	N
Q		S	S	...	S	S	S	S	S	S	S	S	E	S
S		S	S	...	S	S	S	S	S	S	S	S	E	S
E		E	E	...	E	E	*	*	*	<	<	*	S	N

Application: Segmented Scans

Inputs: value array, flag array, associative operator $\oplus$



Can be used to parallelize sparse-matrix vector multiply (SpMV)

Application: Image Processing

- A **summed-area table** is an algorithm/data structure for quickly generating the sum of values in some rectangular subset of a grid.
- Often used in image processing [Crow, 84].
- Computes **prefix sums in both dimensions**, and then **inclusion-exclusion on the corners** to compute the sum within any rectangular area.

$$I(x, y) = \sum_{x' \leq x, y' \leq y} i(x', y')$$

$$A = (x_0, y_0), B = (x_1, y_0), C = (x_0, y_1), D = (x_1, y_1)$$

$$\sum_{x_0 < x \leq x_1, y_0 < y \leq y_1} i(x, y) = I(D) + I(A) - I(B) - I(C)$$

1.

31	2	4	33	5	36
12	26	9	10	29	25
13	17	21	22	20	18
24	23	15	16	14	19
30	8	28	27	11	7
1	35	34	3	32	6

2.

31	33	37	70	75	111
43	71	84	127	161	222
56	101	135	200	254	333
80	148	197	278	346	444
110	186	263	371	450	555
111	222	333	444	555	666

$$15 + 16 + 14 + 28 + 27 + 11 = 101 + 450 - 254 - 186 = 111$$

Application: Image Processing

- A **summed-area table** is an algorithm/data structure for quickly generating the sum of values in some rectangular subset of a grid.
- Often used in image processing [Crow, 84].
- Computes **prefix sums in both dimensions**, and then **inclusion-exclusion on the corners** to compute the sum within any rectangular area.

$$I(x, y) = \sum_{x' \leq x, y' \leq y} i(x', y')$$

$$A = (x_0, y_0), B = (x_1, y_0), C = (x_0, y_1), D = (x_1, y_1)$$

$$\sum_{x_0 < x \leq x_1, y_0 < y \leq y_1} i(x, y) = I(D) + I(A) - I(B) - I(C)$$

Requires inverse

1.

31	2	4	33	5	36
12	26	9	10	29	25
13	17	21	22	20	18
24	23	15	16	14	19
30	8	28	27	11	7
1	35	34	3	32	6

2.

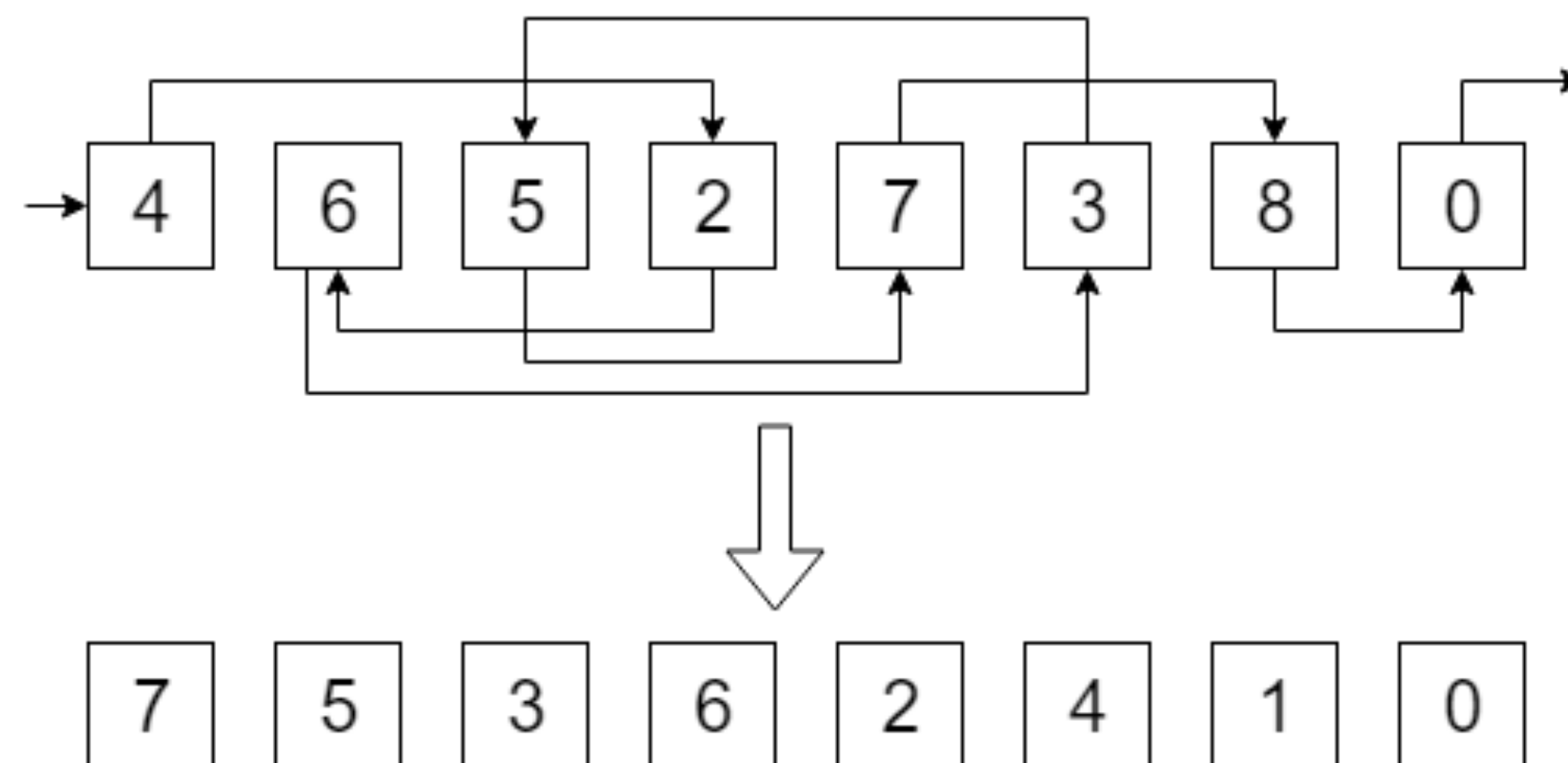
31	33	37	70	75	111
43	71	84	127	161	222
56	101	135	200	254	333
80	148	197	278	346	444
110	186	263	371	450	555
111	222	333	444	555	666

$$15 + 16 + 14 + 28 + 27 + 11 = 101 + 450 - 254 - 186 = 111$$

https://en.wikipedia.org/wiki/Summed-area_table

Application: List Ranking

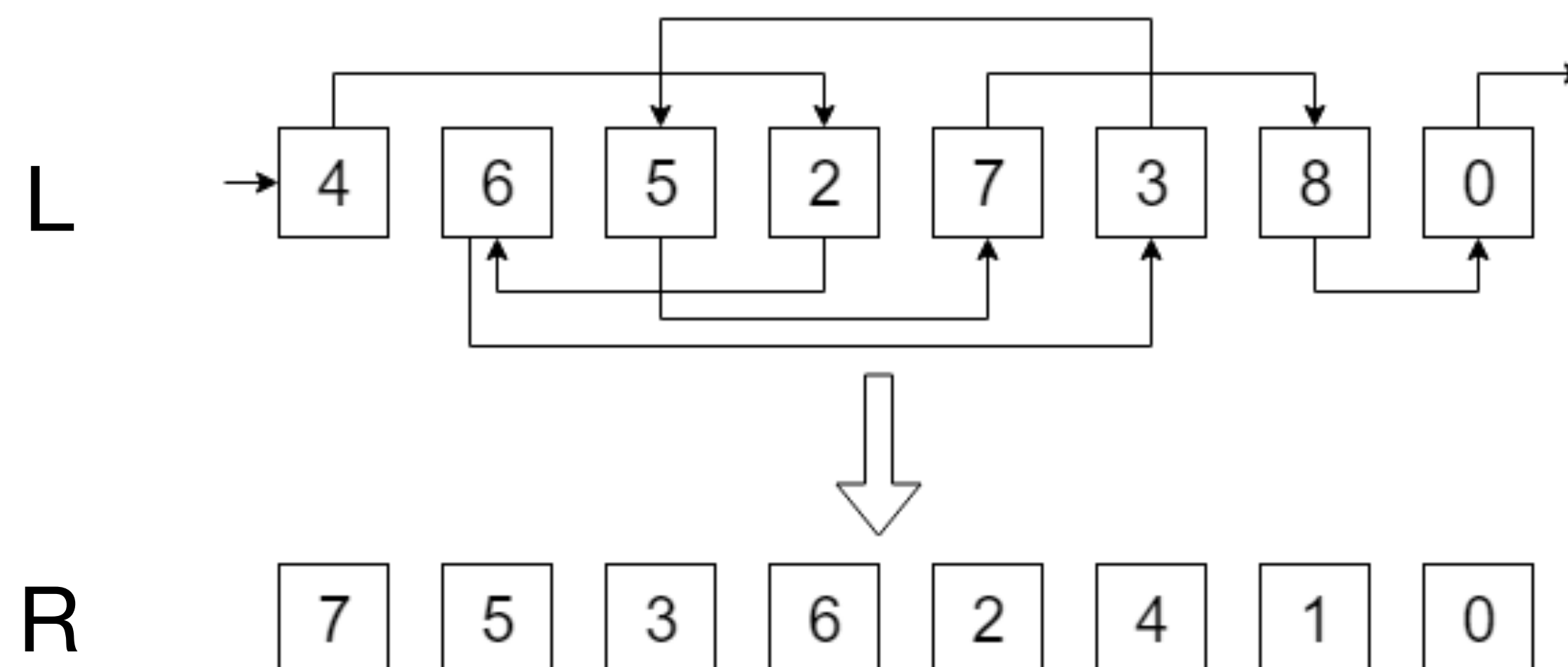
- The list ranking problem is to determine the **distance from every node in a singly-linked list to the end**.
- In serial, the problem can be solved by traversing the list in $O(n)$ time.
- List ranking underlies many applications like prefix sum over linked lists, and tree algorithms.



Application: List Ranking

Input: A linked list L of n nodes with an array S specifying their order. That is, every entry in the list L is a pointer to the index of its successor (or 0, if it is the end node).

Output: A list R of length n containing the rank (distance from the end of the list) of each element.

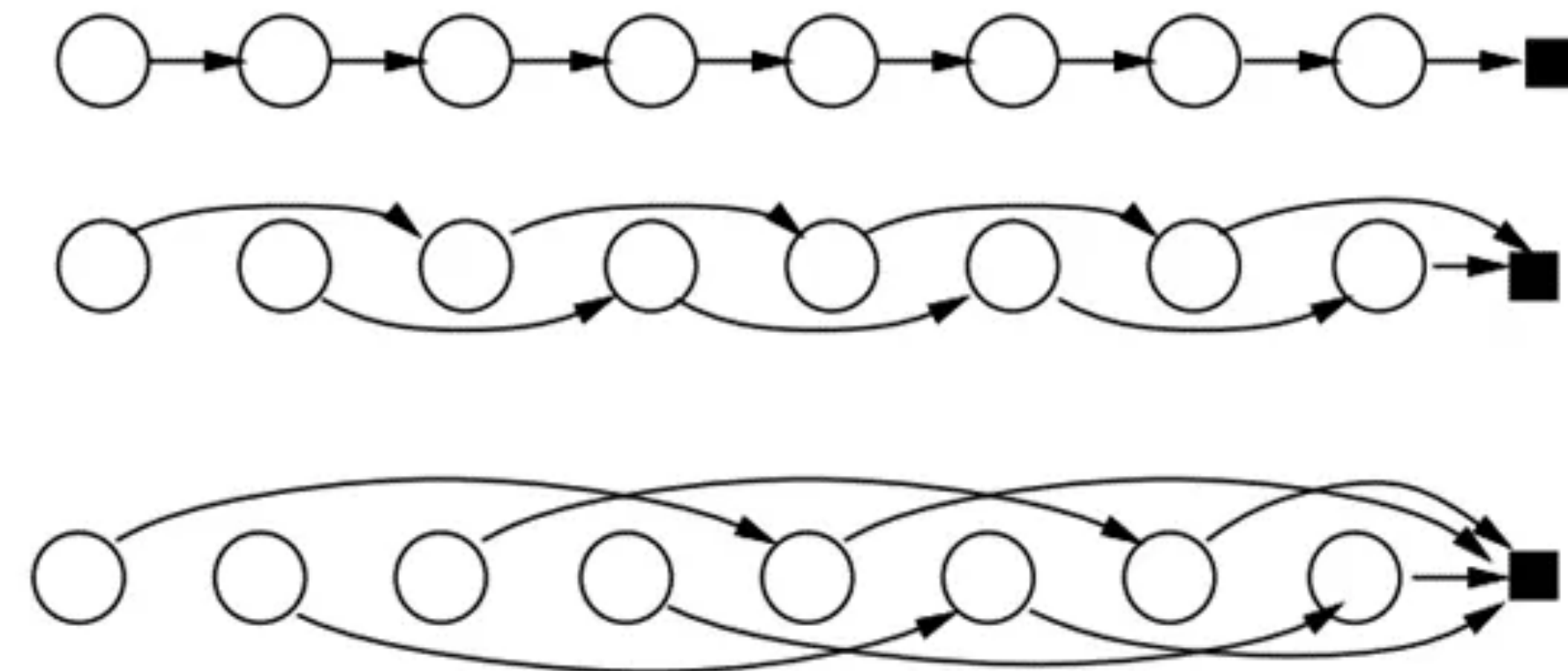


Wyllie's Algorithm for List Ranking

L = list of input successor pointers in an array
 R = output list of ranks (distance from end)

```
parallel_for i = 0 to n-1:
  if L[i] != 0: R[i] = 1
  else: R[i] = 0
```

```
for j = 0 to ceil(lg n) - 1:
  temp, temp2;
  parallel_for i = 0 to n-1:
    temp[i] = R[L[i]];
    temp2[i] = L[L[i]];
  parallel_for i = 0 to n-1:
    R[i] = R[i] + temp[i];
    L[i] = temp2[i];
```

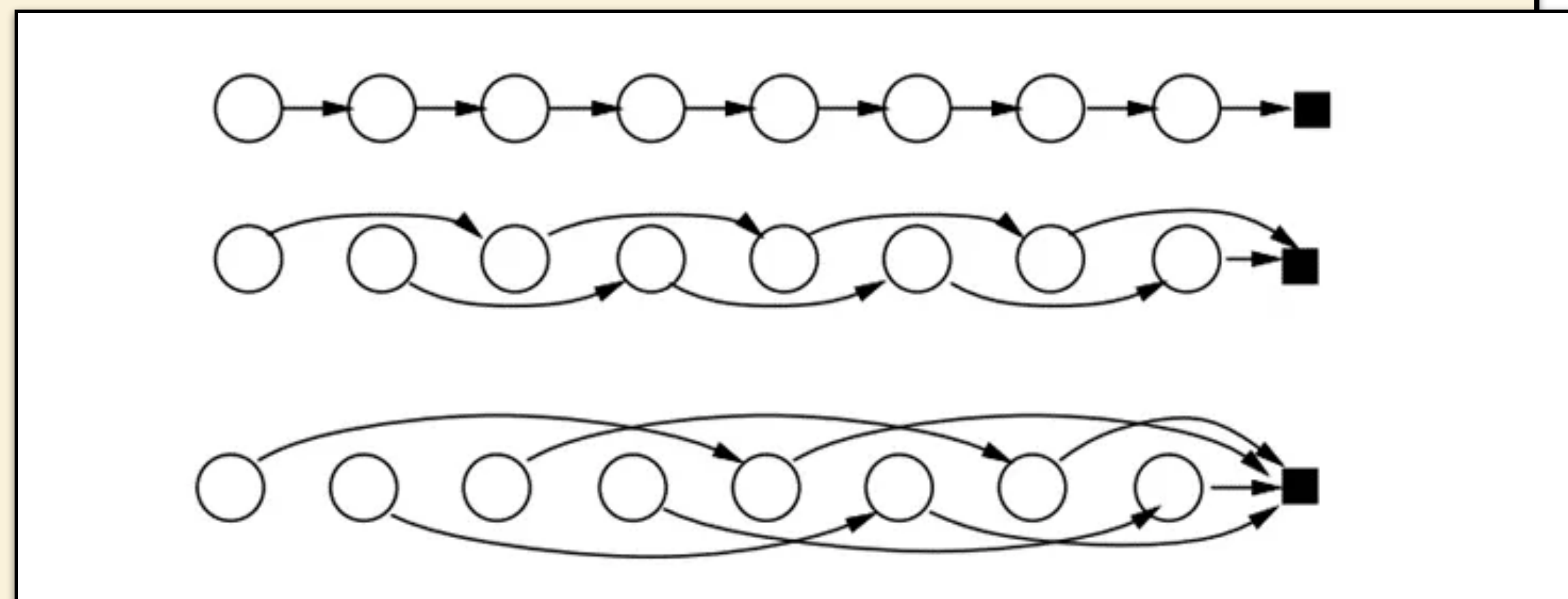


Wyllie's Algorithm for List Ranking

L = list of input successor pointers in an array
 R = output list of ranks (distance from end)

```
parallel_for i = 0 to n-1:
  if  $L[i] \neq 0$ :  $R[i] = 1$ 
  else:  $R[i] = 0$ 
```

```
for j = 0 to  $\text{ceil}(\lg n) - 1$ :
  temp, temp2;
  parallel_for i = 0 to n-1:
     $\text{temp}[i] = R[L[i]]$ ;
     $\text{temp2}[i] = L[L[i]]$ ;
  parallel_for i = 0 to n-1:
     $R[i] = R[i] + \text{temp}[i]$ ;
     $L[i] = \text{temp2}[i]$ ;
```



What is the work and the span?

Work-Span Analysis

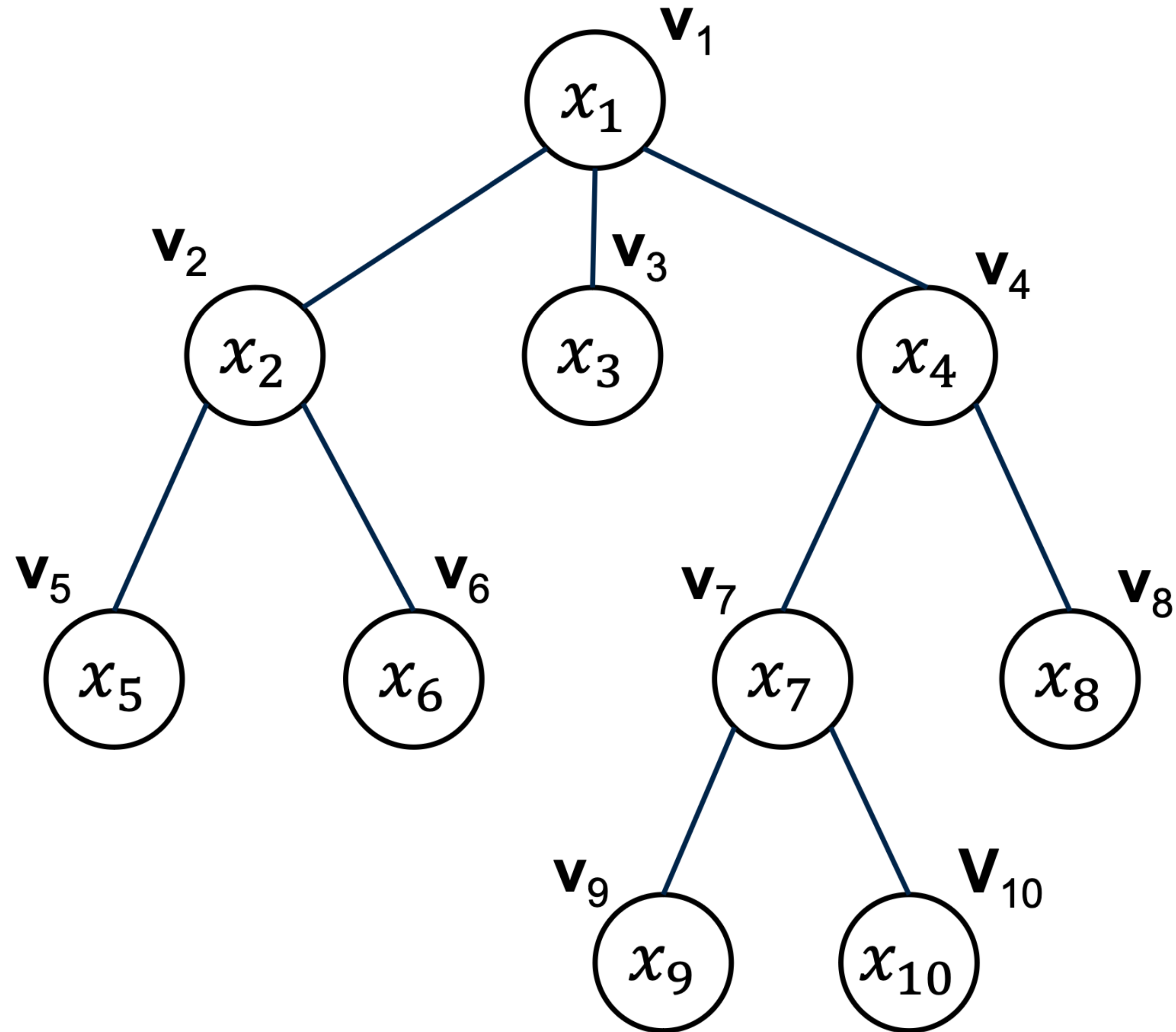
L = list of input successor pointers in an array
R = output list of ranks (distance from end)

```
parallel_for i = 0 to n-1:  
  if L[i] != 0: R[i] = 1  
  else: R[i] = 0  
  
for j = 0 to ceil(lg n) - 1:  
  temp, temp2;  
  parallel_for i = 0 to n-1:  
    temp[i] = R[L[i]];  
    temp2[i] = L[L[i]];  
  parallel_for i = 0 to n-1:  
    R[i] = R[i] + temp[i];  
    L[i] = temp2[i];
```

Not work-efficient:
sequential algorithm only
requires $O(n)$ work

Work = $O(n \log n)$
Span = $O(\log^2 n)$

Application: Tree Accumulation



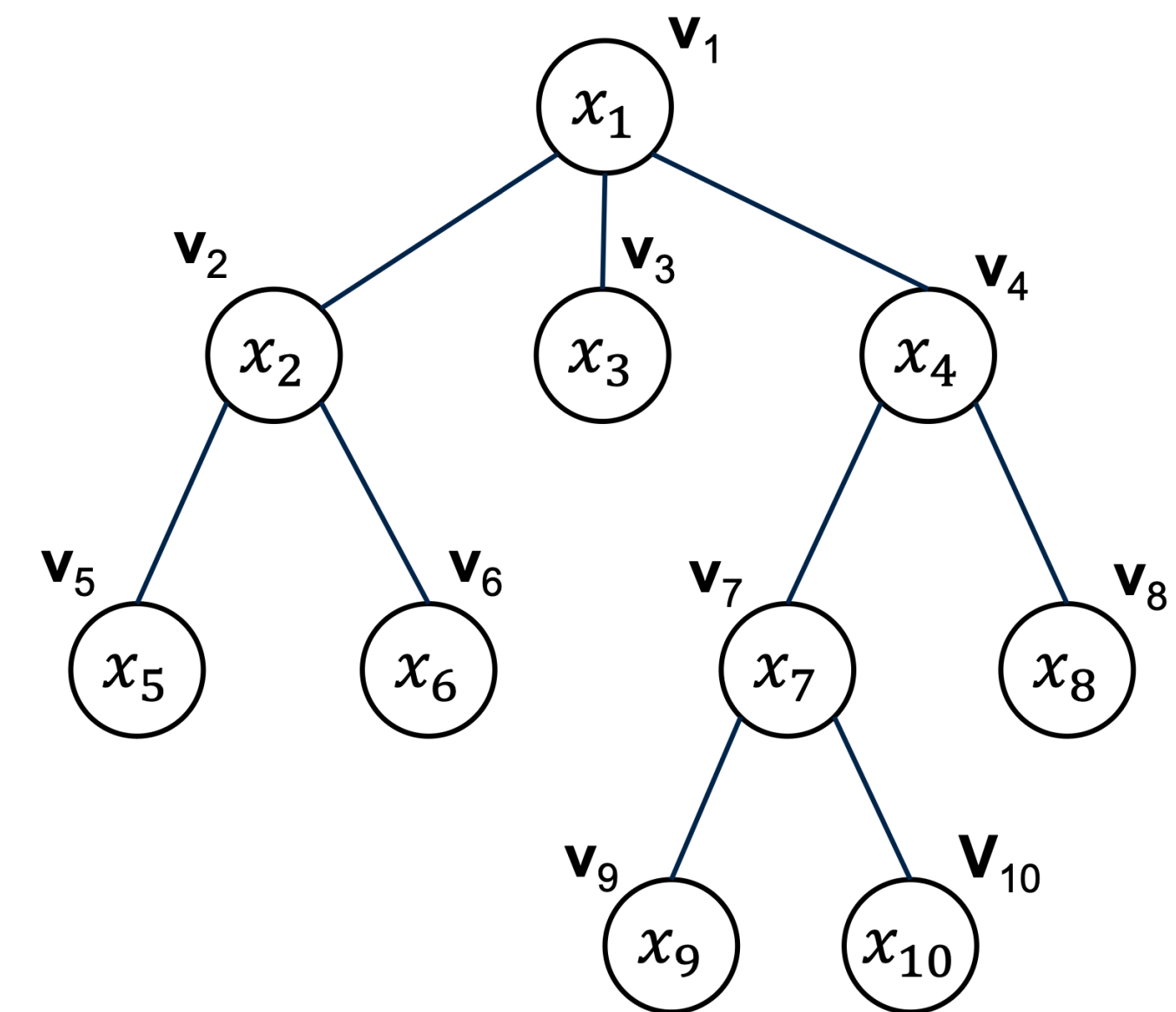
Application: Tree Accumulation

Upward Accumulation (UA): For a node v , sum all numbers in the subtree

- Example: $UA(v_4) = x_4 + x_7 + x_8 + x_9 + x_{10}$
- We want to compute it **for all nodes**

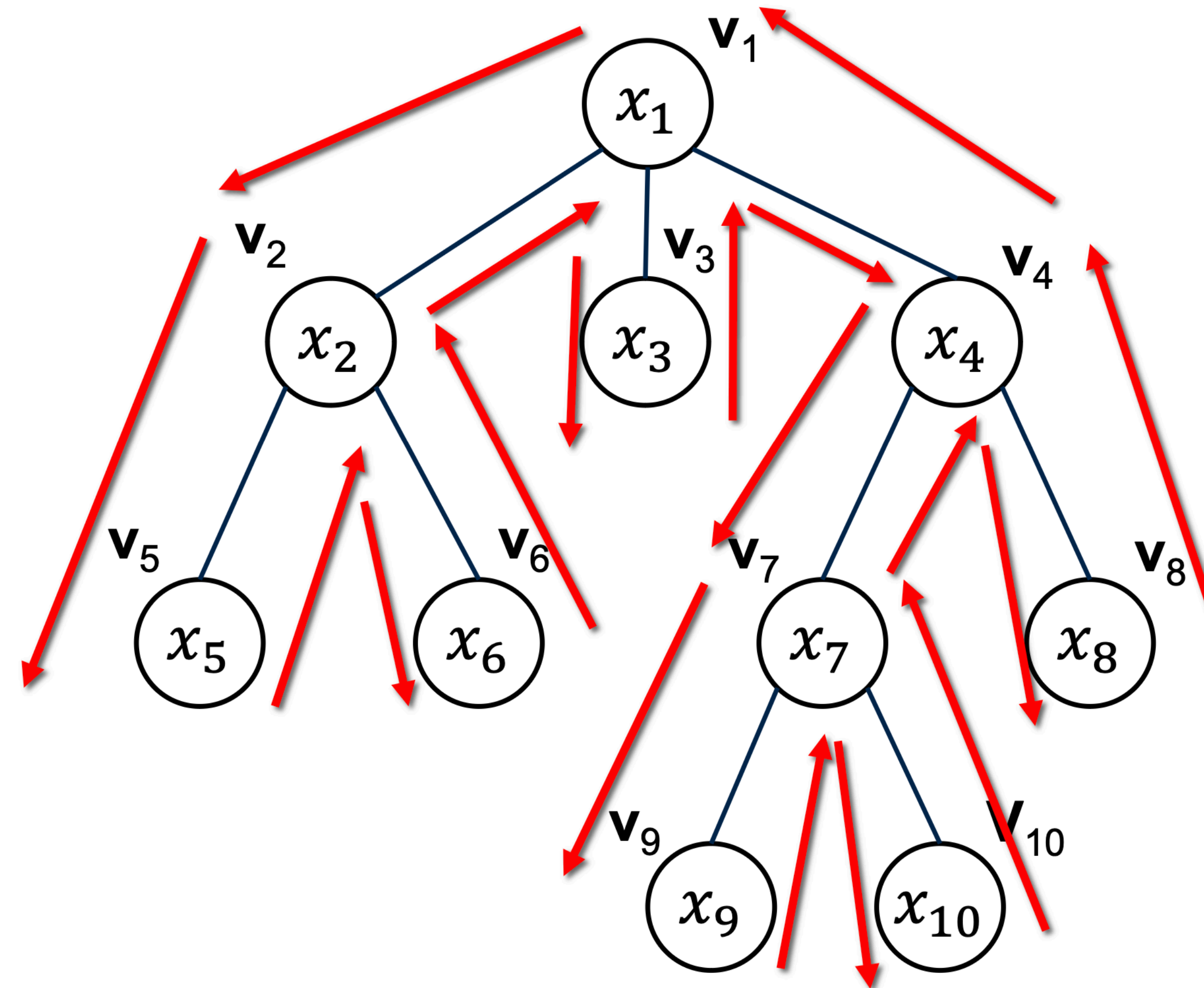
Downward Accumulation (DA): For each node v , sum all numbers in ancestors of v

- Example: $DA(v_7) = x_7 + x_4 + x_1$



Serial computation time is $\Theta(n)$ for both UA and DA

Euler Tour Example

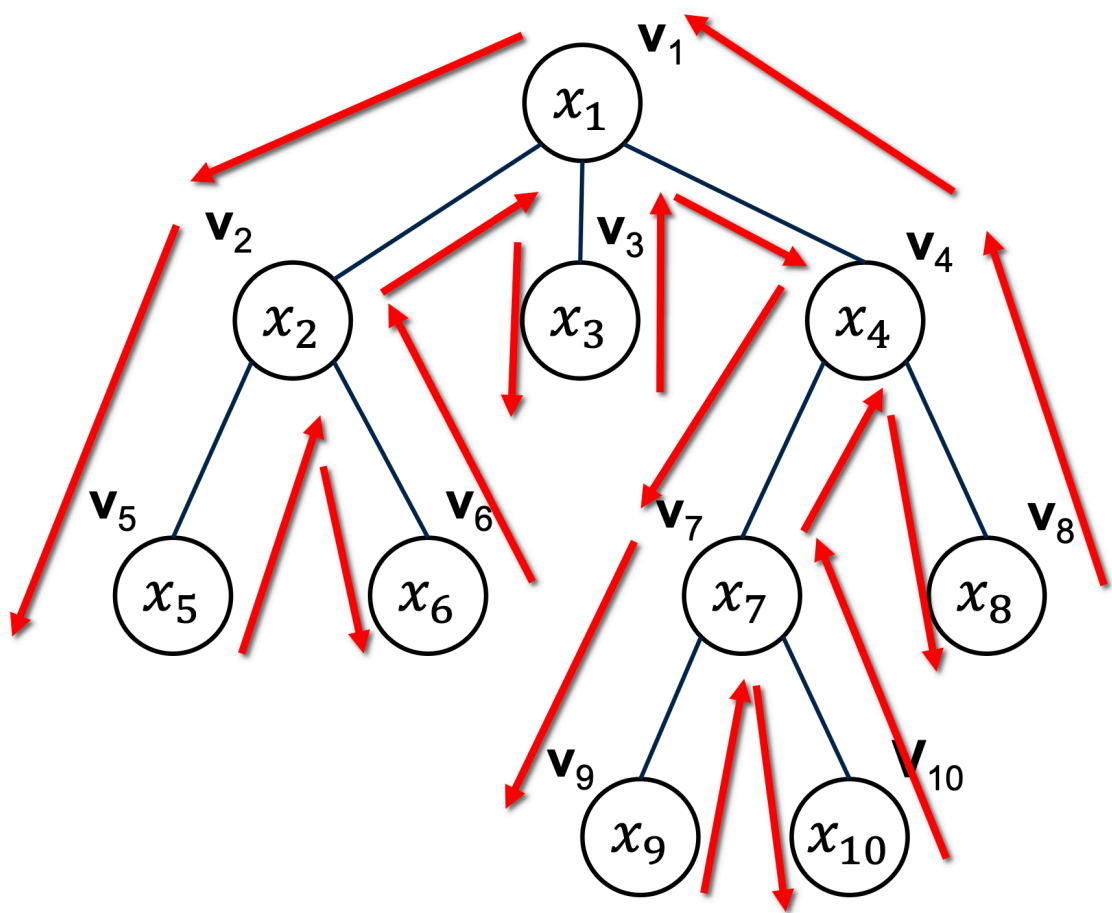


Euler Tour: $v_1 v_2 v_5 v_2 v_6 v_2 v_1 v_3 v_1 v_4 v_7 v_9 v_7 v_{10} v_7 v_4 v_8 v_4 v_1$

Tree Accumulation: UA

Sum in subtree

- ET: $v_1 v_2 v_5 v_2 v_6 v_2 v_1 v_3 v_1 v_4 v_7 v_9 v_7 v_{10} v_7 v_4 v_8 v_4 v_1$
- $|ET| = 1 + 2(n - 1) = 2n - 1$
- Assume ET is block partitioned among processors.
 - First occurrence of v_i put x_i
 - Otherwise put 0



Euler Tour: $v_1 v_2 v_5 v_2 v_6 v_2 v_1 v_3 v_1 v_4 v_7 v_9 v_7 v_{10} v_7 v_4 v_8 v_4 v_1$

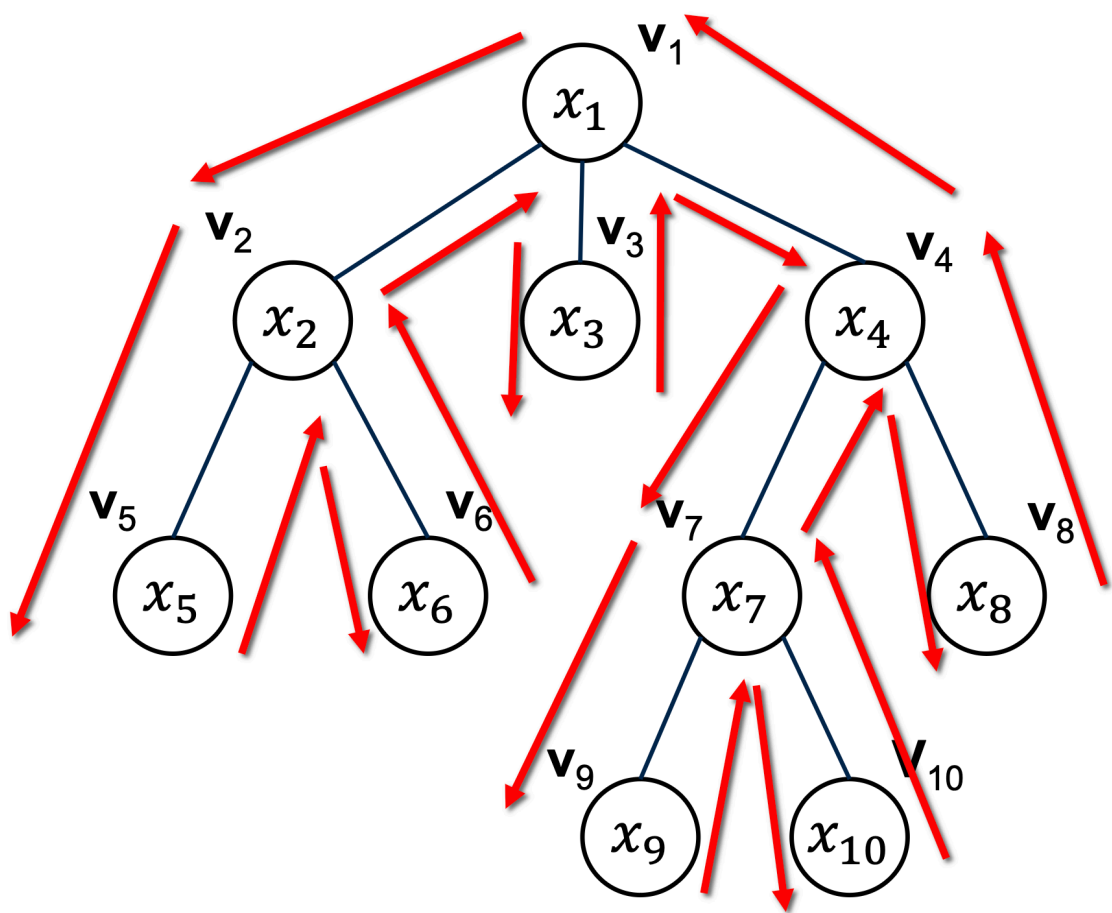
v_1	v_2	v_5	v_2	v_6	v_2	v_1	v_3	v_1	v_4	v_7	v_9	v_7	v_{10}	v_7	v_4	v_8	v_4	v_1
x_1	x_2	x_5	0	x_6	0	0	x_3	0	x_4	x_7	x_9	0	x_{10}	0	0	x_8	0	0

- PS: resulting prefix sums array
- i_f : index of first occurrence of v
- i_l : index of last occurrence of v
- $UA(v) = PS[i_l] - PS[i_f - 1]$

Tree Accumulation: UA

Sum in subtree

- ET: $v_1 v_2 v_5 v_2 v_6 v_2 v_1 v_3 v_1 v_4 v_7 v_9 v_7 v_{10} v_7 v_4 v_8 v_4 v_1$
- $|ET| = 1 + 2(n - 1) = 2n - 1$
- Assume ET is block partitioned among processors.
 - First occurrence of v_i put x_i
 - Otherwise put 0



Euler Tour: $v_1 v_2 v_5 v_2 v_6 v_2 v_1 v_3 v_1 v_4 v_7 v_9 v_7 v_{10} v_7 v_4 v_8 v_4 v_1$

v_1	v_2	v_5	v_2	v_6	v_2	v_1	v_3	v_1	v_4	v_7	v_9	v_7	v_{10}	v_7	v_4	v_8	v_4	v_1
x_1	x_2	x_5	0	x_6	0	0	x_3	0	x_4	x_7	x_9	0	x_{10}	0	0	x_8	0	0

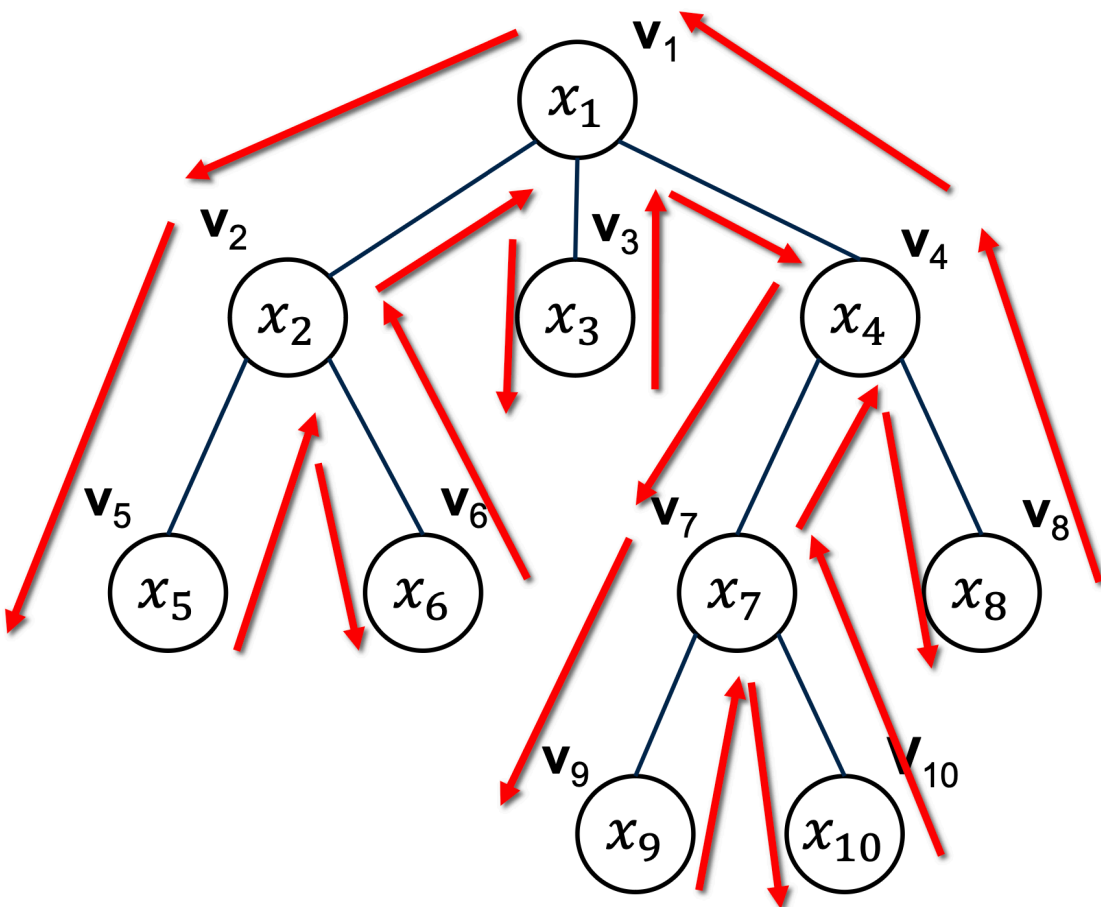
- PS: resulting prefix sums array
- i_f : index of first occurrence of v
- i_l : index of last occurrence of v
- $UA(v) = PS[i_l] - PS[i_f - 1]$

How do we tell if an entry is the first occurrence? We need to find it in parallel and each one in $O(1)$

Tree Accumulation: UA

Sum in subtree

- ET: $v_1 v_2 v_5 v_2 v_6 v_2 v_1 v_3 v_1 v_4 v_7 v_9 v_7 v_{10} v_7 v_4 v_8 v_4 v_1$
- $|ET| = 1 + 2(n - 1) = 2n - 1$
- Assume ET is block partitioned among processors.
 - First occurrence of v_i put x_i
 - Otherwise put 0



Euler Tour: $v_1 v_2 v_5 v_2 v_6 v_2 v_1 v_3 v_1 v_4 v_7 v_9 v_7 v_{10} v_7 v_4 v_8 v_4 v_1$

v_1	v_2	v_5	v_2	v_6	v_2	v_1	v_3	v_1	v_4	v_7	v_9	v_7	v_{10}	v_7	v_4	v_8	v_4	v_1
x_1	x_2	x_5	0	x_6	0	0	x_3	0	x_4	x_7	x_9	0	x_{10}	0	0	x_8	0	0

- PS: resulting prefix sums array
- i_f : index of first occurrence of v
- i_l : index of last occurrence of v
- $UA(v) = PS[i_l] - PS[i_f - 1]$

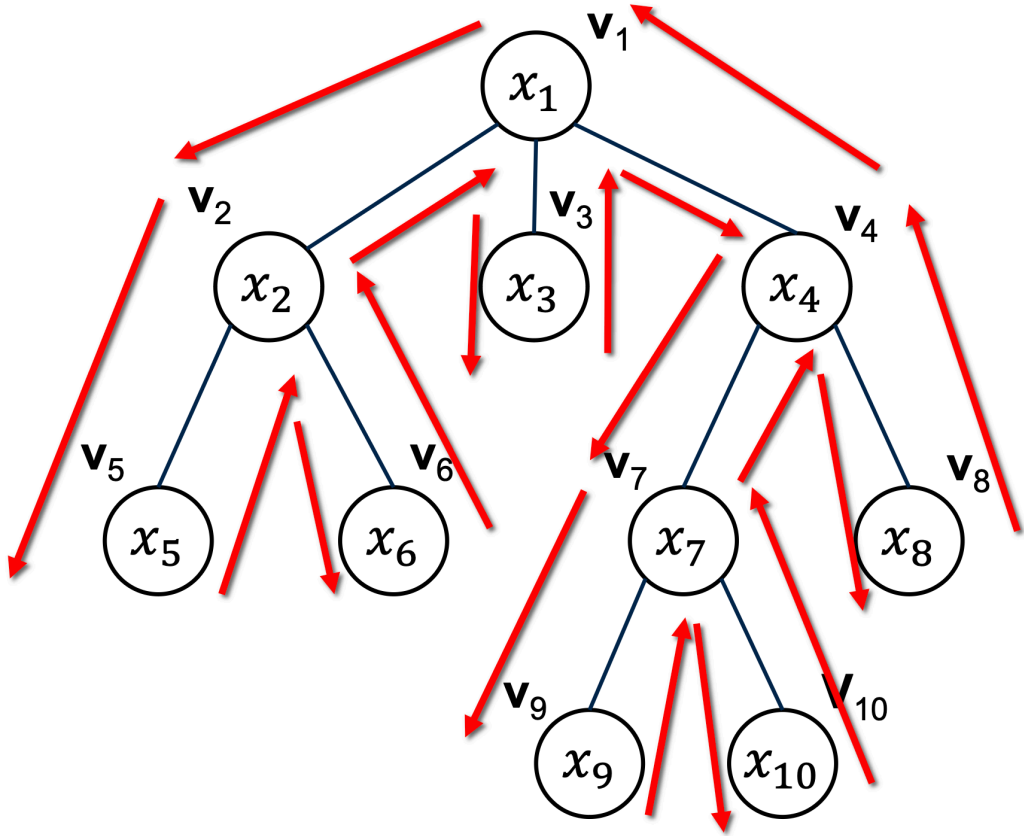
How do we tell if an entry is the first occurrence? We need to find it in parallel and each one in $O(1)$

Look to your left - if it is the parent, you are the first occurrence

Tree Accumulation: DA

Sum up self + ancestors

- ET: $v_1 v_2 v_5 v_2 v_6 v_2 v_1 v_3 v_1 v_4 v_7 v_9 v_7 v_{10} v_7 v_4 v_8 v_4 v_1$
- Assume ET is block partitioned among processors.
 - First occurrence of v_i put x_i
 - Last occurrence of v_i put $-x_i$
 - Otherwise put 0



Euler Tour: $v_1 v_2 v_5 v_2 v_6 v_2 v_1 v_3 v_1 v_4 v_7 v_9 v_7 v_{10} v_7 v_4 v_8 v_4 v_1$

v_1	v_2	v_5	v_2	v_6	v_2	v_1	v_3	v_1	v_4	v_7	v_9	v_7	v_{10}	v_7	v_4	v_8	v_4	v_1
x_1	x_2	x_5	0	x_6	$-x_2$	0	x_3	0	x_4	x_7	x_9	0	x_{10}	$-x_7$	0	x_8	$-x_4$	$-x_1$

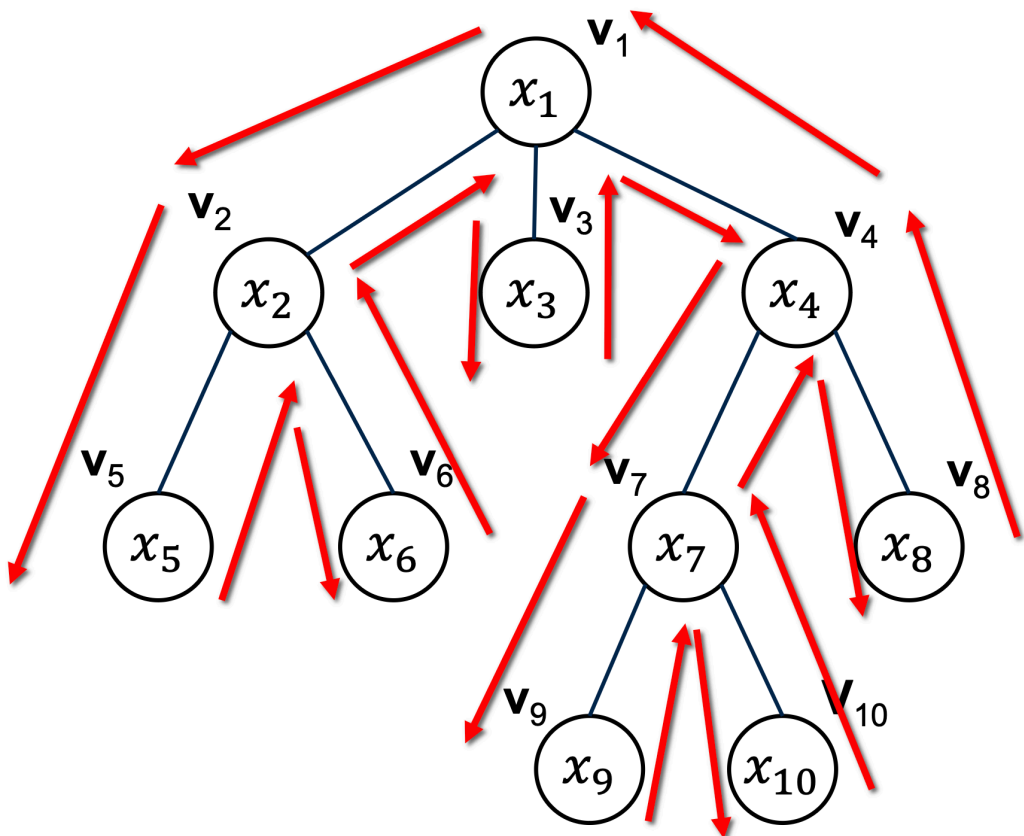
- PS: resulting prefix sums array
- i_f : index of first occurrence of v
- $DA(v) = PS[i_f]$

How do we know which is the last occurrence? We need to find it in parallel and each one in $O(1)$

Tree Accumulation: DA

Sum up self + ancestors

- ET: $v_1 v_2 v_5 v_2 v_6 v_2 v_1 v_3 v_1 v_4 v_7 v_9 v_7 v_{10} v_7 v_4 v_8 v_4 v_1$
- Assume ET is block partitioned among processors.
 - First occurrence of v_i put x_i
 - Last occurrence of v_i put $-x_i$
 - Otherwise put 0



Euler Tour: $v_1 v_2 v_5 v_2 v_6 v_2 v_1 v_3 v_1 v_4 v_7 v_9 v_7 v_{10} v_7 v_4 v_8 v_4 v_1$

v_1	v_2	v_5	v_2	v_6	v_2	v_1	v_3	v_1	v_4	v_7	v_9	v_7	v_{10}	v_7	v_4	v_8	v_4	v_1
x_1	x_2	x_5	0	x_6	$-x_2$	0	x_3	0	x_4	0	x_9	0	x_{10}	$-x_7$	0	x_8	$-x_4$	$-x_1$

- PS: resulting prefix sums array
- i_f : index of first occurrence of v
- $DA(v) = PS[i_f]$

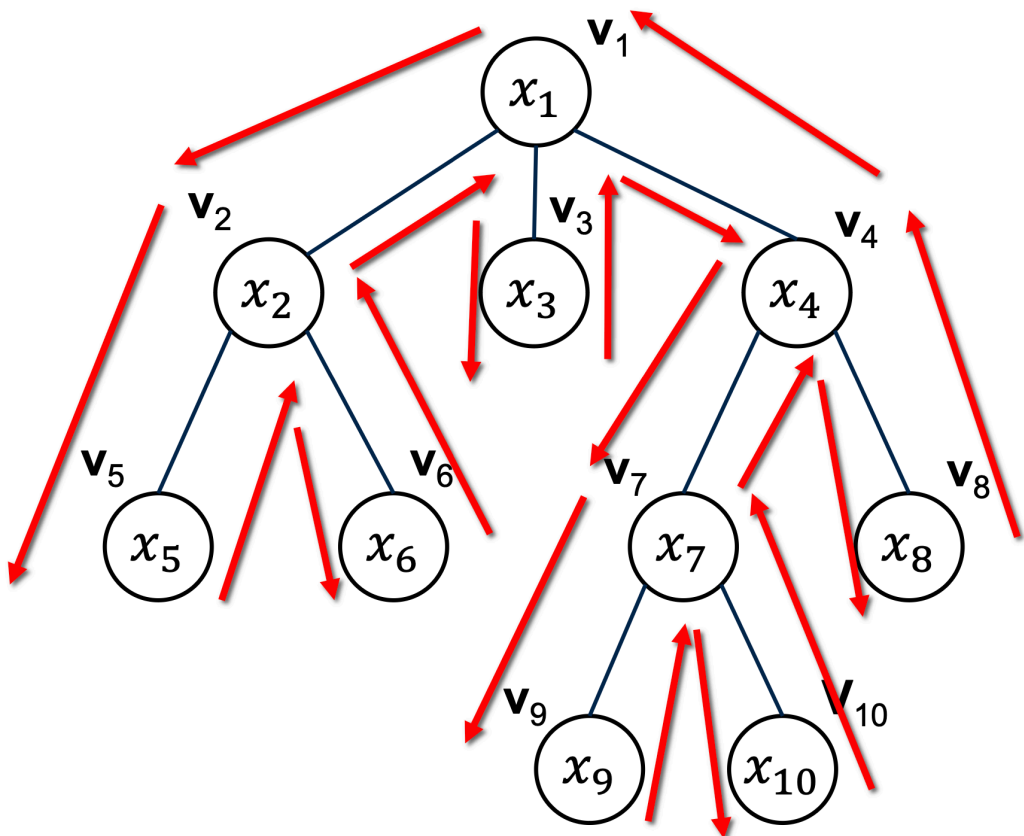
How do we know which is the last occurrence? We need to find it in parallel and each one in $O(1)$

Look to your right - if it is the parent, you are the last occurrence

Tree Accumulation: DA

Sum up self + ancestors

- ET: $v_1 v_2 v_5 v_2 v_6 v_2 v_1 v_3 v_1 v_4 v_7 v_9 v_7 v_{10} v_7 v_4 v_8 v_4 v_1$
- Assume ET is block partitioned among processors.
 - First occurrence of v_i put x_i
 - Last occurrence of v_i put $-x_i$
 - Otherwise put 0



Euler Tour: $v_1 v_2 v_5 v_2 v_6 v_2 v_1 v_3 v_1 v_4 v_7 v_9 v_7 v_{10} v_7 v_4 v_8 v_4 v_1$

v_1	v_2	v_5	v_2	v_6	v_2	v_1	v_3	v_1	v_4	v_7	v_9	v_7	v_{10}	v_7	v_4	v_8	v_4	v_1
x_1	x_2	x_5	0	x_6	$-x_2$	0	x_3	0	x_4	x_7	x_9	0	x_{10}	$-x_7$	0	x_8	$-x_4$	$-x_1$

- PS: resulting prefix sums array
- i_f : index of first occurrence of v
- $DA(v) = PS[i_f]$

How do we know which is the last occurrence? We need to find it in parallel and each one in $O(1)$

Is this correct? Hint: Check the leaves

Look to your right - if it is the parent, you are the last occurrence

Tree Accumulation: DA

Sum up self +
ancestors

- ET: $v_1 v_2 v_5 v_2 v_6 v_2 v_1 v_3 v_1 v_4 v_7 v_9 v_7 v_{10} v_7 v_4 v_8 v_4 v_1$
- Assume ET is block partitioned among processors.
 - Internal nodes:
 - First occurrence of v_i put x_i
 - Last occurrence of v_i put $-x_i$
 - Otherwise put 0
 - Leaf nodes: put 0

v_1	v_2	v_5	v_2	v_6	v_2	v_1	v_3	v_1	v_4	v_7	v_9	v_7	v_{10}	v_7	v_4	v_8	v_4	v_1
x_1	x_2	0	0	0	$-x_2$	0	0	0	x_4	x_7	0	0	0	$-x_7$	0	0	$-x_4$	$-x_1$

- PS: resulting prefix sums array
- i_f : index of first occurrence of v
- If v_i is an internal node
 - $DA(v_i) = PS[i_f]$
- If v_i is a leaf node
 - $DA(v_i) = PS[i_f] + x_i$

Summary

- Prefix sums have applications in across integer and string processing.
- They may be used in spatial data (eg 2D, 3D, etc).
- Although they look serial at first, they admit efficient parallel algorithms.
- When doing prefix sums problems: specify the operator (must be binary associative) and the data inputs and outputs - how does prefix sum get to the desired result?